niagaraRestApi v1 — User Guide VERSION 1

Module: niagaraRestApi (Niagara 4.14) · **Default port:** 8081 · **Vendor:** Shanghai Gline Net Co.

> ■ **This document covers version 1 only.** v1 is the free, unauthenticated module shipped in the public repository. If you need authentication, BQL, program management, or facet editing, see the separate **v2 guide**.

Table of Contents

1. Overview & Architecture
2. Authentication — None
3. REST API Endpoints
   3.1 GET /api/status
   3.2 GET /api/modules
   3.3 GET /api/types
   3.4 GET /api/config
   3.5 POST /api/batch (10 Operations)
4. Verified Test Samples
5. Deployment
6. Known Limitations

1. Overview & Architecture

niagaraRestApi is a custom Niagara N4 module that embeds an HTTP server inside a Niagara Station, exposing a JSON REST API for programmatic interaction — without requiring SMA or OEM licenses.

Architecture

```
RestApiService  (extends BComponent)
 ├─ enabled  (boolean)
 ├─ port     (int, default 8081)
 ├─ started() → Creates HttpServer
 └─ stopped() → Stops  HttpServer

Handlers:
  Status  → GET  /api/status
  Modules → GET  /api/modules
  Types   → GET  /api/types?module=xxx
  Config  → GET  /api/config?path=/xxx
  Batch   → POST /api/batch
```

- JDK built-in HttpServer — no external container.
- JSON-only request/response.
- **No authentication** — designed for internal LAN use only.
- Batch mutation model: all writes go through `POST /api/batch`.

2. Authentication — None

v1 has **no authentication at all**. Every request is accepted without credentials.

```
# No credentials needed on any endpoint
curl http://<HOST>:8081/api/status
{"status":"ok","name":"niagaraRestApi","port":8081}

curl -X POST http://<HOST>:8081/api/batch \
  -H "Content-Type: application/json" -d '[{"OP":"list","PATH":"/"}]'
```

> ■ **Do NOT expose port 8081 to the public internet.** There is no credential check. For production, restrict access with firewall rules or place a reverse proxy (nginx/Apache) with HTTP Basic Auth in front of it.

3. REST API Endpoints Reference

3.1 GET /api/status

Check if the REST API service is running.

```
curl http://<HOST>:8081/api/status
{"status":"ok","name":"niagaraRestApi","port":8081}
```

3.2 GET /api/modules

List all loaded modules with their type counts. **Fully functional in v1.**

```
curl http://<HOST>:8081/api/modules
[{"name":"baja","types":347},{"name":"modbusCore","types":77},
 {"name":"modbusTcp","types":7},{"name":"niagaraRestApi","types":1}]
```

3.3 GET /api/types

Get all component types for a given module. **Fully functional in v1.**

```
curl "http://<HOST>:8081/api/types?module=control"
[{"type":"control:NumericWritable","module":"control"},
 {"type":"control:BooleanWritable","module":"control"}, ...]
```

Common Module Types

Module	Common Types	Description
baja	Folder	Container
control	NumericWritable, BooleanWritable, EnumWritable	Point types
modbusTcp	ModbusTcpNetwork, ModbusTcpDevice	TCP network/device
modbusCore	NumericProxyExt, BooleanProxyExt	Modbus proxy extension
kitControl	Add, Ramp, SineWave, Random	Simulation signals

3.4 GET /api/config

Browse the station configuration tree at any path.

```
curl "http://<HOST>:8081/api/config?path=/"
{"name":"station","type":"Station",
 "slots":{"sysInfo":{"type":"sysInfo","val":"","w":true},
          "Services":{"type":"Services","val":"Service Container","w":true}, ...}}
```

In v1 the `slots` member is an **object** keyed by slot name, each with `type` / `val` / `w` (writable).

3.5 POST /api/batch — Core Operation

All mutations go through this endpoint. Send a JSON array of operation objects executed sequentially in order.

Common Fields

Field	Type	Description	Applies To
OP	string	Operation type	ALL
PATH	string	Component path	ALL
T	string	Type module:Type	add
N	string	Name	add, mod, link
NEW	string	New name	rename, cp
V	any	Value	mod
TARGET	string	Target path	link, unlink
TARGET_N	string	Target slot	link
VAL	any	Invoke value	invoke
VAL_TYPE	string	num / bool / str	invoke
TARGET_PATH	string	Copy destination	cp

10 Operations

Op	Action	Example
1. ADD	Create component	`{"OP":"add","PATH":"/","T":"baja:Folder","N":"myFolder"}`
2. LINK	Connect	`{"OP":"link","PATH":"/src","N":"out","TARGET":"/tgt","TARGET_N":"in10"}`
3. RENAME	Rename (preserves links)	`{"OP":"rename","PATH":"/folder/old","NEW":"new"}`
4. CP	Deep copy + rebuild links	`{"OP":"cp","PATH":"/folder","NEW":"copy","TARGET_PATH":"/"}`
5. MOD	Write property	`{"OP":"mod","PATH":"/device","N":"ipAddress","V":"<IP>"}`
6. UNLINK	Remove link	`{"OP":"unlink","TARGET":"/tgtPoint"}`
7. REMOVE	Delete component	`{"OP":"remove","PATH":"/parent/child"}`
8. READ	Read property	`{"OP":"read","PATH":"/device","N":"ipAddress"}`
9. LIST	List children	`{"OP":"list","PATH":"/myFolder"}`
10. INVOKE	Call action	`{"OP":"invoke","PATH":"/point/set","VAL":"42","VAL_TYPE":"num"}`

MOD Special Values

Type	JSON Format	Example

String	`V:"value"`	`V:"<IP>"`
Integer	`V:number`	`V:502`
Boolean	`V:true/false`	`V:true`
FlexAddress	`V:"addr|format"`	`V:"2|decimal"`

- Enum types (regType, dataType) cannot be written via MOD (BString-to-enum conversion fails).
- BFacets types (deviceFacets) cannot be written via MOD.

4. Verified Test Samples

All operations were tested on a Niagara 4.14 station with default admin credentials.

Modbus TCP Device

```
curl -X POST http://<HOST>:8081/api/batch -H "Content-Type: application/json" -d '[
{"OP":"add","PATH":"/Drivers/ModbusTcpNetwork","T":"modbusTcp:ModbusTcpDevice","N":"T8600_new"},
{"OP":"mod","PATH":"/.../T8600_new","N":"deviceAddress","V":2},
{"OP":"mod","PATH":"/.../T8600_new","N":"ipAddress","V":"<IP>"},
{"OP":"add","PATH":"/.../T8600_new/points","T":"control:NumericWritable","N":"OnOff"},
{"OP":"add","PATH":"/.../OnOff","T":"modbusCore:ModbusClientNumericProxyExt","N":"proxyExt"},
{"OP":"mod","PATH":"/.../OnOff/proxyExt","N":"dataAddress","V":"2|decimal"},
{"OP":"invoke","PATH":"/Drivers/ModbusTcpNetwork/T8600_new/ping"}]'
```

Add + Link + Rename

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"testFolder"},
 {"OP":"add","PATH":"/testFolder","T":"control:NumericWritable","N":"src"}, ...]
```

Result: the link survives the rename (ORD uses a handle, not the name).

Deep Copy (CP)

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"original"},
 {"OP":"add","PATH":"/original","T":"control:NumericWritable","N":"point1"}, ...]
```

Result: internal links are rebuilt in the copy.

Full Lifecycle

```
[{"OP":"add","PATH":"/","T":"baja:Folder","N":"lifecycle"}, ...]
```

Result: unlink removes the Link child; remove deletes the source.

5. Deployment

1. Copy `niagaraRestApi-rt.jar` into the station host's `modules/` directory.
2. Restart the station so the module loads.
3. The REST API is then live on the configured port (default **8081**).

Dependencies

baja 4.14 · nre 4.14 · control 4.14 · modbusCore 4.14 · modbusTcp 4.14

6. Known Limitations

Limitation	Detail
No Authentication	Any client on the network can call the API. Restrict at the network layer.
Enum Properties	regType, dataType: cannot write via MOD (BString→enum fails).
BFacets Properties	deviceFacets: cannot write via MOD.
No BQL	BQL is a v2 feature. On v1 the request returns `{"OP":"bql","status":"unknown"}`.
No Program Endpoints	`/api/program/*` does not exist on v1 (HTTP 404).
No Facet Editing	Facet writes are a v2.4 feature.
No Alarm/History Query	The REST API has no alarm or history endpoints. Use the station's built-in XML API or its Web Box feature (covered in the v2 guide).

For commercial customization, integration, or deployment support, please contact:

Shanghai Gline Net Co.
Email: jason.zhang@gline-net.com
Tel / WhatsApp: +86 138 0199 0968

© 2026 Shanghai Gline Net Co.